CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

Rep Rank

Team	Team 03, section .003
Document version	1.0
Date	11/08/2026
Repository URL	https://github.com/AkshayaKomaragiri/software-engineering-project
Git tag for this milestone	p2
Submitted on Canvas by	Anshika Anshika

Team member	NetID	GitHub username
Laxmi Komaragiri	lak230000	AkshayaKomaragiri
Veda Manchana	vxm230056	veda-manchana
Anshika Anshika	fxa240020	Anshika-111
Shreya Singaravel Sabapathi	sxs230570	shreyasing12
Tanishq Seetala	txs240002	stanishq1205
David Wittler	dnw220000	BestDProgrammer or David-Wittler

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.1	10/17/2026	Tanishq Seetala	P1 baseline (2.1 and 2.6)
0.2	11/2/2026	Laxmi Komaragiri	Updated sections 2.2, 3.1, 4.1 and 3.2 based on P1 instructor feedback.
1.0	11/8/2026	Team 03	P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Name	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Laxmi Komaragiri	3.1, 3.2, 2.2, 4.1, 4.2, 5.2	docs/elicitation **dev/p2-contribution-laxmi** p2-laxmi-contributions.md PR # 19: https://github.com/AkshayaKomaragiri/software-engineering-project/pull/19	PR #13 https://github.com/AkshayaKomaragiri/software-engineering-project/pull/13 PR # 22 https://github.com/AkshayaKomaragiri/software-engineering-project/pull/22#pullrequestreview-5820631160
Veda Manchana	6.1, 9.1, 9.2, 11	PR #17: https://github.com/AkshayaKomaragiri/software-engineering-project/pull/17	PR #13: https://github.com/AkshayaKomaragiri/software-engineering-project/pull/13 PR #16: https://github.com/AkshayaKomaragiri/software-engineering-project/pull/16
Anshika Anshika	0.2, 4.1(revised), 4.2(revised), 4.3, 10.3, 10.4	PR#12, PR#13	PR#20
Tanishq Seetala	6.3-6.6 and both UML figure	AkshayaKomaragiri/software-engineering-project at dev/p2-documentation-tanishq	PR #13: https://github.com/AkshayaKomaragiri/software-engineering-project/pull/13
David Wittler	10.1, 10.2, 8, 5.3	S - 01 Security Validation · Issue #15 · AkshayaKomaragiri/software-engineering-project p2-David-contributions.md dev/spike/flutter_supabase_sync	PR #19 :Dev/p2 documentation laxmi by AkshayaKomaragiri · Pull Request #19 · AkshayaKomaragiri/software-engineering-project

Shreya Singaravel Sabapathi	Section 6.2 , 7.1 ,7.2	PR #18 : https://github.com/AkshayaKomaragiri/software-engineering-project/pull/18 PR # 22 : https://github.com/AkshayaKomaragiri/software-engineering-project/pull/22	PR #14 : https://github.com/AkshayaKomaragiri/software-engineering-project/pull/14

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	FR-04 now states the estimated 1RM formula, weight × (1 + reps/30), and its acceptance criterion checks values against it	P1 feedback: finalize the 1RM formula so FR-04 can be tested	Progress component (6.3) and FR-04 row in 9.1 use this formula	Nov 2, 2026
CH-02	QR-02 now defines standard network conditions as 307–314 Mbps, measured with a speed-test tool	P1 feedback: define “standard network conditions”	QAS-01 (6.1) refers to this definition	Nov 2, 2026
CH-03	Strength Training Users row in 2.2 now based on two gym-user interviews held Nov 1 (docs/elicitation)	P1 feedback: needs were stated as assumptions; interviews would make the need more convincing	2.6 operational verdict and the UC-02 selection reason (4.1) cite the interviews	Nov 2, 2026
CH-04	Updated section 10.1 working agreement to specify team meeting times and communication channel	P1 feedback: specify meeting arrangements and communication method.	Clarifies team coordination for P2 and P3.	Nov 2, 2026
CH-05	Merged P1 documentation branches into main, as recorded in §10.4.	P1 feedback: merge individual documentation branches.	Makes team documentation accessible from the main branch.	Nov 2, 2026

CH-06	Used Flutter/SQLite prototype and S-01 results as architectural evidence in §§7–8.	P1 feedback: support architectural decisions with prototype evidence.	Supports ADR-01, ADR-02, and ADR-03.	Nov 8, 2026
CH-07	Added a brief comparison of Rep Rank with existing workout-tracking applications.	P1 feedback: compare Rep Rank with existing workout-tracking applications.	Reinforces the initial scope and exclusion of competitive leaderboards in §1.2.	Nov 7, 2026

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

This document is version 1.0 of the Rep Rank specification. It carries forward the P1 requirements and adds the analysis and high-level design for Rep Rank, a mobile application focused on personal fitness consistency and workout tracking. The document outlines the problem the application aims to address, identifies its intended users and stakeholders, and defines the system's scope. It also describes the functional and quality requirements, identifies the constraints that must be considered during development, and outlines the external services and dependencies the application will rely on. The document serves as a reference for the development team, course instructor, and stakeholders to evaluate the proposed system, and establish a shared understanding of its objectives.

Rep Rank is designed to help individuals who want to maintain consistent workout routines and monitor their fitness progress in a convenient way. The application aims to help users who may struggle to maintain a consistent workout routine, remember previous workouts, or keep track of their fitness goals. The application will allow users to search for exercises, log exercise details such as sets, repetitions, and weights, review previous workout records, and view exercise progress through line graphs. Users will also be able to create accounts and access their saved workout records across devices. The main goal of Rep Rank is to help users stay consistent with their workouts and track their personal progress over time.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Search for exercises by name using a search bar	Building our own cloud storage infrastructure, since we will use a third-party service
Log workouts by adding exercises and recording sets, repetitions, and weights	Building our own user authentication service, since we will use a third-party provider
View exercise progress over time through a line graph	Health diagnosis and medical advice, since Rep Rank is a fitness tracking app
Create an account and log in	Workout scheduling and calendars, since the initial version focuses on logging workouts rather than planning them.
View workout progress through a personal dashboard displaying exercise progress graphs and workout history.	Exercise rankings, competitive leaderboards, and weight-class competitions, since they introduce additional complexity and rely on user-reported exercise performance.
Access saved workout records across devices using the same account.	Wearable device integration requires additional development and testing.
Set weekly workout goals between one and seven sessions (COULD - optional).	

We will use third-party services for cloud storage and user authentication because we do not have the resources or infrastructure to build and maintain these services ourselves. Medical advice, workout scheduling, calendars, competitive leaderboards, and wearable integration are excluded to keep Rep Rank focused on workout tracking and personal progress visualization.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Rep (repetition)	One movement that completes an exercise, such as one squat, or one deadlift.
Set	A collection of reps performed together. A set of 8 reps for squat means the person did a complete squat movement 8 times. 3 sets of 8 means the person did this 3 times.
MAU (Monthly Active Users)	The number of unique users who use the application at least once during a calendar month.
Workout	A completed workout that may include multiple exercises and sets. Session = “workout” (Session in the domain model)
Workout History	The set of a user’s workouts
Exercise log	A record of one exercise performed in a workout, with weight, sets and repetitions (Exercise Entry in the domain model)
Weekly workout goal	Number of workouts a user aims to complete within a calendar week.
Progress dashboard	A screen that displays the user's workout history and exercise progress through line graphs.
Exercise performance	The weights, sets, and repetitions recorded for a particular exercise during previous workouts.
1RM (One-Rep Max)	Heaviest weight one can lift for a single, proper-form repetition of a specific exercise
Progress graph	A graph of One- Rep Max over time (One-Rep Max on y-axis and date on x-axis)
CRUD	C – create, R – read, U – update, D - delete

1.4 References

[R1] Federal Trade Commission, “Mobile Health Apps Interactive Tool.” [Online]. Available: https://www.ftc.gov/business-guidance/resources/mobile-health-apps-interactive-tool. [Accessed: Oct. 16, 2026].

[R2] StatCounter GlobalStats, “Mobile Operating System Market Share in the United States.” [Online]. Available: https://gs.statcounter.com/os-market-share/mobile/united-states-of-america. [Accessed: Oct. 16, 2026].

[R3] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: Oct. 16, 2026].

[R4] International Organization for Standardization and International Electrotechnical Commission, Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model, ISO/IEC 25010:2023, 2023. [Online]. Available: https://www.iso.org/standard/78176.html. [Accessed: Oct. 16, 2026].

[R5] T. J. McCabe, “A complexity measure,” IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308–320, Dec. 1976, doi: 10.1109/TSE.1976.233837. [Online]. Available: https://doi.org/10.1109/TSE.1976.233837. [Accessed:

Oct. 16, 2026].

[R6] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.2, Success Criterion 1.3.4: Orientation.” [Online]. Available: https://www.w3.org/TR/WCAG22/#orientation. [Accessed: Oct. 16, 2026].

[R7] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.2, Success Criterion 2.5.8: Target Size (Minimum).” [Online]. Available: https://www.w3.org/TR/WCAG22/#target-size-minimum. [Accessed: Oct. 16, 2026].

[R8] Federal Trade Commission, “Federal Trade Commission Act.” [Online]. Available: https://www.ftc.gov/legal-library/browse/statutes/federal-trade-commission-act. [Accessed: Oct. 16, 2026].

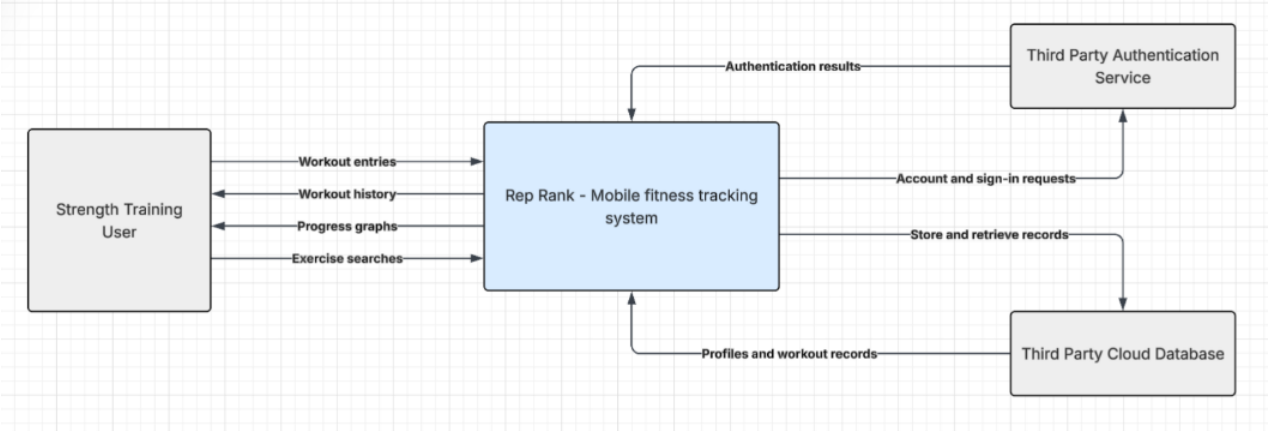
[R9] Centers for Disease Control and Prevention, “Adding Physical Activity as an Adult,” Dec. 4, 2025. [Online]. Available: https://www.cdc.gov/physical-activity-basics/adding-adults/. [Accessed: Oct. 17, 2026].

[R10] Supabase, “Pricing.” [Online]. Available: https://supabase.com/pricing. [Accessed: Oct. 18, 2026].

[R11] Intuit Mailchimp, “Transactional Email Pricing.” [Online]. Available: https://mailchimp.com/pricing/transactional-email/. [Accessed: Oct. 18, 2026].

2 Overall Description

2.1 Product perspective and context



Rep Rank is a mobile fitness tracking application that enables strength training users to search for exercises, record workouts, review previous records, and monitor progress through graphs. The system boundary includes the application interface, workout processing, progress calculations, local storage, and synchronization logic. Third-party authentication and cloud database services remain outside this boundary because external providers supply their infrastructure. Rep Rank connects to these services to authenticate users and save and retrieve account-specific records across devices. The team is responsible for configuring and testing these connections, protecting access to user records, and handling service failures. The exercise catalogue is bundled with the application, so no external catalogue service is involved.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Strength Training Users	Primary Users of Rep Rank	They need a convenient way to record their exercises, review previous workout sessions, and monitor changes in their exercise performance over time.	Interviewed Nov 1st 2026, See docs/elicitation
Development Team	System designers, developers, and maintenance teams.	They require clearly defined requirements, manageable development scope, and maintainable application.	This was identified through the team's project planning and discussions regarding development responsibilities.
Third-party cloud and authentication providers	External services used for account authentication and data storage.	They require compatible integration, correct usage of services, and compliance with service requirements.	This was identified from the decision to outsource authentication and cloud storage in Section 1.2. No formal provider consultation documented.
App distribution platforms	External stakeholders responsible for hosting and distributing the application.	They require applications to meet the applicable platform requirements for installation and documentation.	This is an assumption based on the decision to support iOS and Android users. The specific platform requirements have not yet been formally reviewed.
Federal Trade Commission and relevant privacy authorities	Regulatory stakeholders are responsible for enforcing consumer protection and privacy requirements.	They require protection of user data privacy and truthful representation of the application's purpose and its capabilities.	This was identified through reviewing the FTC Mobile Health App Interactive Tool and Federal Trade Commission Act [R1, R8].

2.3 User characteristics

The primary users of Rep Rank are individuals who want to maintain consistent workout routines and track their personal fitness progress. These users include beginners who are developing regular exercise habits and more experienced individuals who want to organize their workout records and monitor their progress over time.

Beginner users are expected to have basic knowledge of mobile applications, but they may not be familiar with fitness terminology such as exercises, sets, reps (repetitions), one-rep max, and weights. They should not be expected to understand the application's internal data storage or authentication processes. Rep Rank should use clear labels, readable text, and simple navigation to help beginners search for exercises, record workout details, and understand their previous exercise performance.

Experienced users may already be familiar with strength-training terminology and regularly track their workout performance. These users may want to quickly review previous exercise records, compare weights across sessions, and view progress through line graphs on the progress dashboard. Since workout information is entered by the users, the accuracy of their records and progress graphs will depend on the information they enter.

Both user groups will primarily use iOS and Android mobile devices, often in gym environments. Users may record exercises during short breaks or while tired or distracted, so common actions like searching for exercises and entering sets, repetitions, and weights should be quick and easy to perform. Users may also have different accessibility needs, so the application should provide readable text, clear labels, and simple navigation.

Comparison with Existing Workout-Tracking Applications: Existing workout-tracking applications such as Strong and Hevy offer features for recording exercises, reviewing workout history, and monitoring strength-training progress. Rep Rank focuses on similar core needs, including recording weights, sets, and repetitions, viewing previous performance, and visualizing estimated one-repetition maximum (1RM) trends. Rather than competing through social features or leaderboards, Rep Rank prioritizes a focused workout-logging experience, local workout storage, and access to saved records across devices. This comparison supports the decision to keep competitive leaderboards outside the initial project scope.

2.4 Constraints

Rep Rank's design and development are subject to the following constraints.

1. Platform Requirements: The application must support iOS and Android operating systems. These platforms were specifically selected because they represent the majority of the United States mobile operating system market. Development and compatibility support for other operating systems are excluded from the initial release [R2].
2. Privacy and data handling: The application must protect any personal and fitness related information collected from users, including workout records and profile information. Personal workout information must not be publicly disclosed without appropriate user authorization except where disclosure is required by applicable law. This constraint limits how the workout records and personal information may be accessed, stored, or shared with third parties [R1].
3. Truthful health-related claims: Rep Rank must not make any false, misleading or unsupported claims regarding improvements in the users' physical health, strength or fitness. The application is intended for workout tracking and progress visualization rather than medical diagnosis or treatment. This constraint restricts the claims that may appear in the application's interface or promotional materials [R8].
4. External service integration: Based on the team's defined project scope and resource limitations, account authorization and cloud storage will rely on third party services rather than the infrastructure developed and operated by the team. This constraint rules out developing custom authentication and cloud storage infrastructure for the initial release (Section 1.2).

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	The selected third-party database and authentication providers are assumed to maintain appropriate security measures to protect user data against unauthorized access and data loss.	It results in user's personal information and workout records being exposed, compromised, or lost, undermining privacy and user trust in the application.
A2	The selected cloud service provider is assumed to maintain sufficient service availability, data integrity, and backup capabilities to support the applications' intended usage.	The users may not be able to save, retrieve, or synchronize workout records reliably across application sessions and devices.

A3	The targeted audiences are assumed to have access to a compatible iOS or Android device and possess familiarity with navigating mobile applications. Knowledge of specific exercise terminology is not assumed.	The prospective users without compatible devices or basic mobile navigation skills may struggle to access the application or use its workout tracking features effectively.
A4	The users are assumed to accurately record their workout performance, including the weights lifted and number of sets and repetitions completed.	Incorrect or inconsistent workout entries may result in inaccurate workout histories and misleading progress graphs, reducing the reliability of the application's progress tracking functionality.
A5	A sufficiently comprehensive and usable exercise dataset is assumed to be available for the application's exercise search functionality.	The users may be unable to find exercises they perform, preventing the application from fully supporting its intended workout logging functionality.
A6	The selected third-party authentication and cloud services are assumed to provide necessary APIs or SDKs compatible with the application's iOS and Android development framework.	Account creation, authentication and workout synchronization may require a different service provider or changes to the application architecture, potentially delaying development.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible; confirmed by spike S-01 (section 8)	Rep Rank's core requirements cover exercise search, workout logging, saved history, and progress graphs. Authentication and cloud storage infrastructure would be supplied by third-party providers, reducing the infrastructure the team must build. Spike S-01 (section 8) showed account-specific local storage, sign-in, separation between two accounts and two-way synchronization with the cloud service on Windows and an Android emulator. iOS and the full workout schema remain to be tested in P3 (10.2).
Operational	Feasible, conditional on user validation	Sections 2.2 and 2.3 identify beginner and experienced users who need convenient workout logging, access to previous records, and progress tracking during short gym rest periods. These needs are based on interviews of potential users of the app see docs/elicitation branch in the repository. The offline prototype planned in 10.2, milestone 4 will allow representative users to test exercise search, workout entry, and history retrieval. Their feedback will help confirm that the workflow fits their routines and identify improvements before cloud synchronization is introduced.
Schedule	Feasible	The project has been divided into manageable phases with defined deliverables, allowing progress to be monitored throughout development (10.2). Technical capabilities are still in question, but a working fitness tracker will be available throughout the development after phase 3 (offline workout tracker with no unique user support).
Economic / resource	Feasible conditional on revenue scaling with MAU	The estimated operating costs of the application are relatively low during initial development and are expected to scale with the size of the user base. The most significant expense to consider is third party database support and transactional email services as the app scales. Supabase costs range from $0 during early development to $1,000 or

		more per month as monthly active users (MAU), database usage, storage, and bandwidth increase [R10]. Transactional email services also scale with usage, with an estimated cost of approximately $20 per 25,000 emails sent[R11]. Advertising revenue from the app is also expected to scale with the number of monthly active users. At 100,000 MAU, advertising revenue is expected to generate thousands per month, although actual revenue will vary significantly based on user engagement, geographic distribution, ad placement, and monetization strategy. Because the application's major operating expenses scale alongside user growth, while potential advertising revenue also increases with a larger and more active user base, the project is considered economically feasible. Initial operating costs should remain relatively low, with additional infrastructure expenses becoming significant primarily as the application gains users.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one "shall" sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow users to record the weight, number of sets, and number of repetitions performed for each exercise during a workout.	MUST	Interviews 1 and 2, Q3 (2.2)	A tester enters an exercise with a specified weight, number of sets, and repetitions, saves the workout, and verifies that the recorded values match what was entered.
FR-02	The system shall save each exercise log to the user's workout history and retain previously recorded logs across application sessions.	MUST	Interviews 1 and 2, Q3 (2.2)	A tester saves an exercise log, closes and reopens the application, and verifies that the previously saved log is still available in workout history.
FR-03	When a user selects an exercise to log, the system shall display the user's previously recorded performance for that exercise.	MUST	Interviews 1 and 2, Q2 (2.2)	A tester records the same exercise with different weights and repetitions across multiple workouts and verifies that the most recent recorded values appear when selecting that exercise again.
FR-04	The system shall calculate an estimated one-repetition maximum (1RM) using the formula: Weight * (1 + (reps/30)) for a selected exercise using recorded weight and repetitions and display the results over time in a line graph.	MUST	Interview 1 Q1; Interview 2 Q2 (2.2)	A tester records exercise weights and repetitions across three workout dates, opens the progress graph, and verifies that the estimated 1RM values are displayed for the corresponding dates and match the formula: Estimated 1RM = Weight × (1 + Repetitions / 30).
FR-05	The system shall allow users to set a weekly workout goal between one and seven sessions.	COULD	Interview 2 Q1 (2.2); [R9]	A tester sets a weekly goal of four workouts, saves it, and verifies that the displayed goal is four workouts per week.

FR-06	The system shall display a leaderboard ranking participating users by their highest recorded weight for a selected exercise.	WON'T	Interview 2 Q2 (2.2)	A tester records different weights for the same exercise under two participating accounts and verifies that the leaderboard ranks the higher recorded weight first.
FR-07	The system shall allow users to create an account using an email address and a third-party authentication method.	MUST	Scope decision (1.2)	A tester registers using a valid email address through the third-party authentication provider and verifies that the account is successfully created.
FR-08	The system shall authenticate registered users before granting access to their private account information.	MUST	CON-02; [R1]	A tester attempts to access private account information without signing in and verifies that access is denied.
FR-09	The system shall make a user's saved workout records available across devices when signed in to the same account.	SHOULD	Scope decision (1.2)	A tester saves a workout on one device, signs in to the same account on another device, and verifies that the saved workout is available.
FR-10	The system shall allow users to search for exercises by name when creating a workout log.	MUST	Scope decision (1.2); A5	A tester searches for an exercise by name and verifies that a matching exercise appears and can be selected.

3.2 Quality requirements

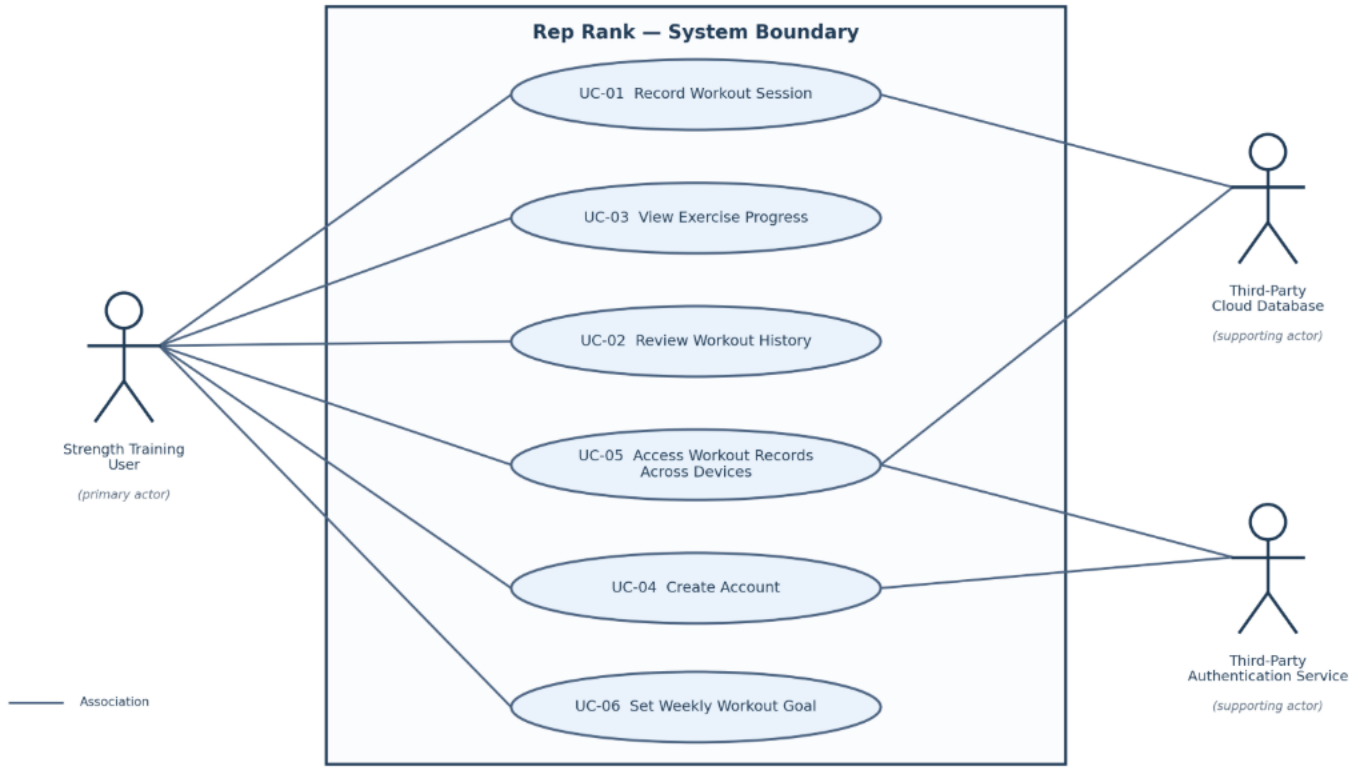
ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	Security – All authentication tokens issued by the third-party provider must be cryptographically verified using the provider's public keys or SDK before granting session access.	MUST	R3	Will be verified with automated unit test that intercepts an authenticated API request using a simulated valid token, invalid token, and expired token, confirming that unauthorized requests are successfully rejected.
QR-02	Performance – The mobile application must achieve a Cold Start time of less than 2.0 seconds under standard network, that is 307 to 314 Mbps, conditions on supported iOS and Android device profiles.	SHOULD	R4	This will be verified using performance measuring tools like Xcode instruments for iOS and android studio profiler on the test devices. The elapsed time from app launch to the first interactive screen for 10 consecutive cold starts will be measured. Additionally, a Wifi speed measurement tool must be used to ensure standard network conditions, which is 307 to 314 Mbps.
QR-03	Maintainability – The codebase should maintain a cyclomatic complexity of no more than 10 per method/function which will be measured through automated static code analysis tools	MUST	R5, R4	This will be verified continuously with automated static code analysis gates such as SonarQube or ESLint/Pylint. It will be integrated into the GitHub Actions CI pipeline

	(such as SonarQube) during code reviews and pull request integration.			which will automatically fail any pull request with a cyclomatic complexity greater than 10.
QR-04	Accessibility – The mobile app must not restrict its view and operation to a single display orientation	COULD	R6	This will be verified manually by inspecting the UI layout, by rotating test devices between portrait and landscape modes across all primary app screens.
QR-05	Accessibility – Pointer and touch targets must be large enough to activate easily. This is 24 by 24 CSS pixels.	COULD	R7	This will be verified using automated accessibility tools, like Google Accessibility Scanner) along with manual UI inspect to confirm that all buttons and touch targets meet or exceed the minimum 24 by 24 CSS pixels threshold.

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system needs to be compatible with IOS and android mobile operating systems	[R2]	More than 99% of all smartphones in the United States use these operating systems. This is essential for accessibility.
CON-02	The application shall maintain the confidentiality of users' personal and health-related information, including age, sex, weight, and workout history. Such information shall not be disclosed to third parties without the user's explicit consent, except where disclosure is required by applicable law.	[R1]	Legal compliance with legislation is mandatory.
CON-03	The application shall not make false, misleading, or unsubstantiated claims regarding its effects on users' health, fitness, or physical performance.	[R8]	Legal compliance with legislation is mandatory

3.4 Initial use cases



3.5 Requirements to use case traceability

Requirement ID	Use case(s) that realize it	If blank — why, and what you will do
FR-01	UC-01	
FR-02	UC-01, UC-02	
FR-03	UC-01	
FR-04	UC-03	
FR-05	UC-06	
FR-06		No use case is written for FR-06. It is a WON'T requirement, and competitive leaderboards are out of scope for the initial release (Section 1.2). The requirement stays in Section 3.1 for traceability. If it is brought back into scope, it will go through change control (Section 0.2) and a use case will be written at that point.
FR-07	UC-04	
FR-08	UC-05	
FR-09	UC-05	
FR-10	UC-01	

The matrix traces nine of the ten functional requirements to at least one of the six use cases. FR-06 is the only requirement with no use case, for the reason given in its row. UC-06 (Set Weekly Workout Goal) was added so that FR-05 has a use case. Every use case in the diagram in Section 3.4 traces to at least one functional requirement, so no use case is left without a requirement.

The matrix also shows three things. FR-02 is realized in two use cases: UC-01 saves the log, and UC-02 retrieves it. FR-08 and FR-09 trace only to UC-05, so UC-05 carries both sign-in and cross-device access. FR-07 traces to UC-04, and UC-04 and UC-05 are the two use cases that use the third-party authentication service.

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-02	Review Workout History	It lets users review past workouts, which both interviewees said they forget or lose track of, and shows that saved records are read back correctly.	(2), (4)
UC-05	Access Workout Records Across Devices	It involves cross-device access. It touches third-party external systems and covers critical security and sign in flows	(4)
UC-01	Record Workout Session	Logging workouts is a core functionality of the app. It deals with primary data processing and involves the most risk and complexity.	(4), (6), (8)

4.2 Expanded use cases

UC-01 Record Workout Session

Primary actor	Strength Training User
Supporting actors and external systems	None
TUCBW (this use case begins when)	The user chooses to record a workout session.
TUCEW (this use case ends when)	The system confirms that the entry was saved successfully.
Preconditions	The user is signed into their Rep Rank account and has opened the application.
Postconditions	The workout entry is saved in the user's workout history and can be retrieved in a later application session.
Traces to requirements	FR- 01, FR-02, FR-03, FR-10

Actor input / action	System response
	(0) The system shows today's workout and offers to record an exercise.
(1) The user opens the workout logging feature.	
	(2) The system asks which exercise to record and offers to find it by name.
(3) The user enters part of the exercise name.	
	(4) The system shows the exercises whose names match.
(5) The user selects an exercise.	

	(6) The system displays the user's most recent recorded performance for that exercise, if available, and asks for today's weights, sets and reps.
(7) The user enters the weight, number of sets, and repetitions performed and saves the workout entry.	
	(8) The system stores the exercise log in the user's workout history and confirms that the entry was saved successfully, showing the saved values.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(4)	The user can't find their specific exercise	If the exercise search returns no matches, the system informs the user and allows another search.	(3)
A2	(6)	No past performance for selected exercise	The system allows the user to continue without displaying past performance.	(7)
A3	(8)	A value is missing or invalid	The system names each field to correct and keeps the other values.	(7)
A4	(8)	The entry cannot be saved	The system says the entry was not saved and keeps the entered values	(7)
A5	After (8)	The user wants to record another exercise	The system shows today's workout with the new entry and asks for the next exercise	(3)

UC-05 Access Workout Records Across Devices

Primary actor	Strength Training User
Supporting actors and external systems	Third-party authentication service and third-party cloud database
TUCBW (this use case begins when)	The user opens Rep Rank on another device to access their saved workout records.
TUCEW (this use case ends when)	The system displays the user's previously recorded workouts.
Preconditions	The user has an existing Rep Rank account. The application is installed on a second supported device
Postconditions	The user is authenticated and can access workout records associated with their account on the second device.
Traces to requirements	FR-08, FR-09, QR-01, CON-02

Actor input / action	System response
	(0) The system shows that no account is signed in on this device, shows no workout records, and offers to sign in.
(1) The user selects the account sign-in options.	
	(2) The system requests the user's account credentials.
(3) The user enters their credentials	
	(4) The system has the third-party authentication service to verify the

	credentials, retrieves the workout records linked to the account from the third-party cloud database, and displays the user's previously recorded workouts.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(4)	The account has no saved workout records	The system displays an empty workout history.	Ends use case
A2	(4)	The user enters incorrect credentials	The system informs the user that the credentials are incorrect	(3)
A3	(4)	The authentication service cannot be reached	The system says sign-in cannot be completed right now.	Ends use case
A4	(4)	The device has no connection	The system says sign-in needs a connection and keeps the entered email	Ends use case
A5	(4)	The cloud database cannot be reached after sign-in	The system says saved records cannot be retrieved right now.	Ends use case
A6	(1)	The user tries to open workout records without signing in	The system shows no records and asks the user to sign in	(1)

UC-02 Review Workout History

Primary actor	Strength Training User
Supporting actors and external systems	None
TUCBW (this use case begins when)	The user selects option to view workout history
TUCEW (this use case ends when)	The user exits the workout history feature.
Preconditions	The user has an existing Rep Rank account and is authenticated.
Postconditions	The user's historical workout sessions are successfully displayed, allow them to review past performance without altering stored data
Traces to requirements	FR-02, FR-08

Actor input / action	System response
	(0) The system shows the main view with the option to view workout history.
(1) The user requests to review past workout records	
	(2) The system displays the user's past workouts, newest first, with their dates.
(3) The user selects a specific past workout for more details	
	(4) The system displays a detailed breakdown of the selected workout, with each exercise's weight, sets and reps.
(5) The user indicates that they are finished reviewing	
	(6) The system closes the historical view and returns to the main interface

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(2)	The user has no previous workouts logged	The system indicated that no workout history is available	Ends use case
A2	(2)	Workouts recorded on another device have not been uploaded on this device	The system shows the workouts available and says recent workouts from other devices may be missing.	(3)
A3	(4)	The user wants to see another workout	The user returns to the list of past workouts.	(2)

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-01	Step (0)	The step (0) row held instruction text instead of what the user sees	Wrote step (0): today's workout and the offer to record an exercise
UC-01	One turn per row	"Searches for and selects" held two actor turns; "enters the values" and "saves" were two actor rows with no system response between them	Split into steps (3) to (5); merged into step (7)
UC-01	No background processing	Step (8) saved to the cloud database, which the user neither sees nor waits for	Removed it from the step; supporting actors set to None
UC-01	No technology	"Search bar", "cloud storage is enabled" and "third-party cloud database" appeared in the steps	Reworded in domain terms
UC-01	Real alternate flows	P1's invalid-input, save-failure and another-exercise flows had been dropped	Restored as A3, A4 and A5
UC-05	Step (0) and complete interaction	Step (0) was blank; the first row was the trigger (TUCBW); the flow went from entering credentials straight to showing workouts	Wrote step (0); removed the trigger row; step (4) now covers verification and retrieval
UC-05	Precondition or alternate flow	"Device has network access" can be false while the system works normally	Network access remains a precondition for successful cross-device sign-in. Alternate flow A4 documents the behavior when a network connection is unavailable.
UC-02	Precondition or alternate flow	"Has previous saved workout records" contradicted A1	Removed from the preconditions
UC-02	No technology; visible response	"Home screen interface" and "cached data locally" were technology; step (2) "retrieves historical data" gave the user nothing to see	Reworded step (0) and A2; step (2) now displays the list
All	Acknowledgment at the end	None: UC-01 step (8), UC-05 step (4) and UC-02 step (6) each end with a response the user sees, matching the TUCEW	No change needed
All	Every actor-column item is a domain attribute	Not yet checkable; the domain model (5.1) is drawn after this review	Items listed below for 5.1; checked again in cross-check K1 (5.3)

5 Analysis Models

5.1 Domain model

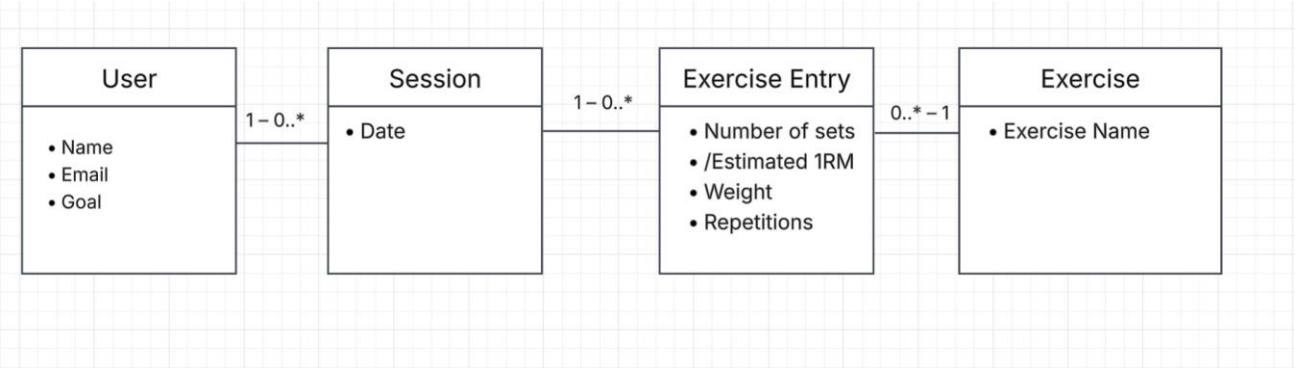


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Exercise	The individual exercise type	Exercise Name	Exercise 1 – 0..* Exercise Entry
Exercise Entry	A record of one exercise performed in a session.	Weight, Repetitions, Number of Sets, /Estimated 1RM (derived)	Session 1 – 0..* Exercise Entry Exercise 1 – 0..* Exercise Entry
Session	A workout performed on a certain date.	Date	Session 1 – 0..* Exercise Entry User 1 – 0..* Session
User	Profile level information	Name, Goal, Email	User 1 – 0..* Session

5.2 Object interaction models

5.2.1 UC-01 Record Workout Session

(a) Non-trivial steps.

- (4) needs the Exercise Catalog to find the exercises whose name matches.
- (6) needs the Workout History to find the most recent Exercise Entry for the chosen exercise.
- (8) checks the values, creates an Exercise Entry, adds to the Workout Session and stores the session in the Workout History.
- Steps (0), (1), (2), (3), (5) and (7) are trivial. (0), (2) and (6)'s prompt only display something. (1), (3), (5) and (7) are inputs. The inputs (3), (5) and (7) still get a row because each one triggers a message.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
3	Strength Training User	Enters part of an exercise name	Name text	Workout Logging Controller
4.1	Workout Logging Controller	Asks for exercises whose names match	Name text	Exercise
4.2	Exercise	Returns the matching exercises	Exercise names	Workout Logging Controller
4.3	Workout Logging Controller	Shows the matching exercises	Exercise names	Strength Training User
5	Strength Training User	Selects an exercise	Chosen exercise	Workout Logging Controller
6.1	Workout Logging Controller	Asks for the user's most recent entry for that exercise	Chosen exercise, user	Session
6.2	Session	Returns its date and the entry's weight, sets and reps	Date, weight, sets, repetitions	Workout Logging Controller
6.3	Workout Logging Controller	Shows the most recent performance and asks for today's values	Date, weight, sets, repetitions	Strength Training User
7	Strength Training User	Enters weight, sets, reps and asks to save	Weight, number of sets, repetitions	Workout Logging Controller
8.1	Workout Logging Controller	Checks that the values are complete and valid	Weight, number of sets, repetitions	Workout Logging Controller
8.2	Workout Logging Controller	Creates an entry with the exercise and the values	Chosen exercise, weight, sets, repetitions	Exercise Entry
8.3	Workout Logging Controller	Adds the entry to the today's session (starting today's session if none exists)	New entry, today's date	Session
8.4	Workout Logging Controller	Asks the user to keep the updated session	Updated session	user
8.5	User(account)	Confirms the session is kept	Updated session	Workout Logging Controller
8.6	Workout Logging Controller	Confirms the entry was saved, showing the saved values	Weight, sets, repetitions	Strength Training User

(c) Analysis sequence diagram.

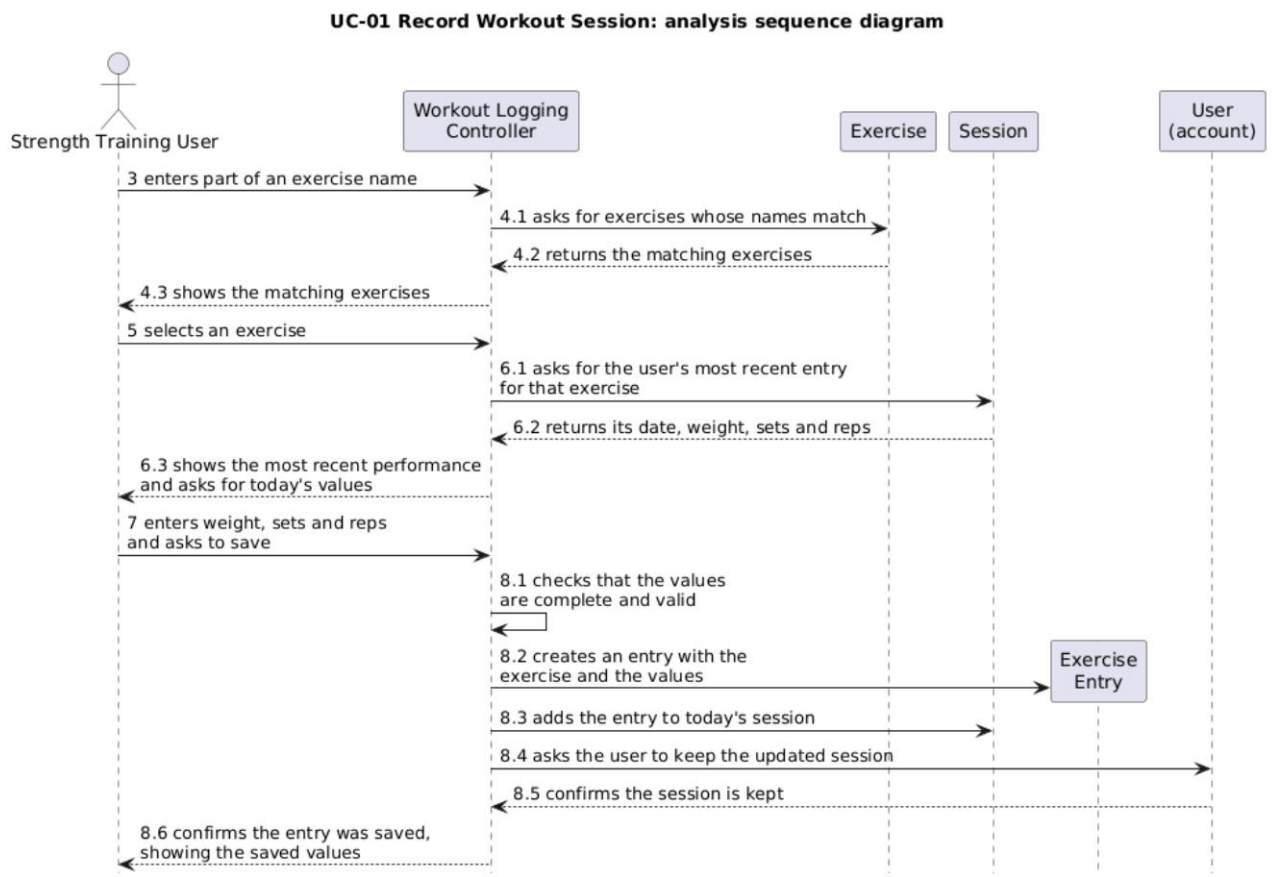


Figure 5.2.1 Analysis sequence diagram for UC-01; message numbers match the scenario table

5.2.2 UC-05 Access Workout Records Across Devices

(a) Non-trivial steps.

- (4) needs Authentication Service to verify the credentials, the Account to record the sign-in, the cloud database to return the account's records, and the Workout History to hold them on this device.
- Steps (0), (1) and (2) are trivial because they only display a state or a prompt, or accept a selection. Step (3) is an input and has a row because it triggers the first message.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
3	Strength Training User	Enters email and password	Email, password	Sign in controller
4.1	Sign in Controller	Asks to verify the email and password	Email, password	Authentication Service
4.2	Authentication Service	Confirms the credentials and identifies the user	Verified identity	Sign in controller
4.3	Sign in Controller	Records that the user is signed in on this device	Verified identity	user
4.4	Sign in Controller	Asks for the user’s saved workouts	Verified identity	Cloud database

4.5	Cloud Database	Returns the user's saved sessions with their entries	Dates, weights, sets, repetitions	Sign in controller
4.6	Sign In Controller	Creates each returned session for the user	Returned sessions and entries	Session
4.7	Sign In Controller	Displays the previously recorded workouts	Session dates	Strength Training User

(c) Analysis sequence diagram.

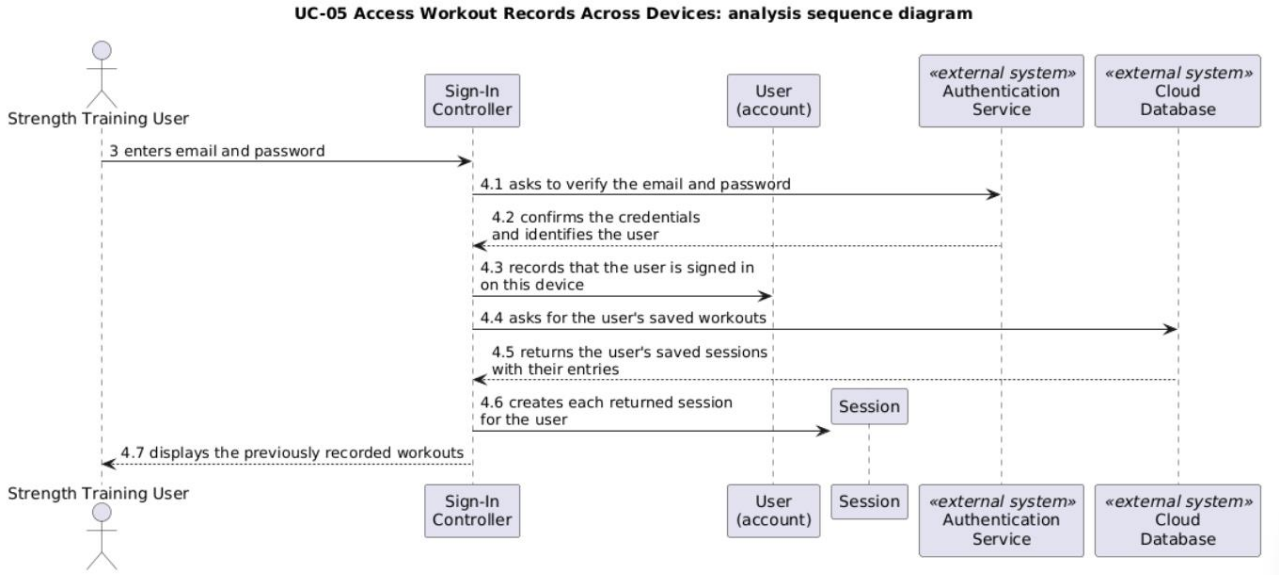


Figure 5.2.2 Analysis sequence diagram for UC-05; message numbers match the scenario table

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	pass	
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Pass For 5.2.1: Workout logging controller is developed in Rep Rank. Exercise, exercise entry, and session are described in 5.1. Strength training user and user(account) are the same entity from different perspectives. For 5.2.2: Sign-in controller is developed in rep rank. Session is described in 5.1. The 3rd party authentication and cloud storage are predetermined.	
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass	
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass	
K5 Every analyzed use case traces to at least one functional requirement	Pass: UC-02 maps to FR-02 UC-05 maps to (FR-08, FR-09) UC-01 maps to (FR-01, FR-02, FR-03, FR-10)	
K6 The last messages of each diagram make the use case postcondition true	5.2.1 UC-01 post condition: " The workout entry is saved in the user's workout history and can be retrieved in a later application session." Partial Pass: There is no arrow describing saving a session onto persistent storage. However, it can be inferred from arrows 6.2 (return most recent entry), 8.3, 8.4, 8.5, and 8.6 (confirming a session is to be saved from the user). 5.2.2 UC-05 Post condition: "The user is authenticated and can access workout records associated with their account on the second device." Pass: 4.5 and 4.7 describe downloading from cloud storage and displaying downloaded entries to the user.	Fixed in 6.2. The chosen architecture confirms that Exercise entry and Session are tables on local device storage, clarifying that adding a session makes it accessible across all application sessions on that device.

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	A user launches Rep Rank from a fully closed state on a supported iOS or Android device under the standard network conditions defined in QR-02.	Flutter mobile application	The application loads and displays its first interactive screen.	Cold-start time is less than 2.0 seconds, measured across 10 consecutive cold starts on each tested device profile under the network conditions defined in QR-02.	QR-02
QAS-02	Security	An unauthenticated user, or a user with an invalid or expired authentication token, attempts to access private workout records.	Auth Gateway and Supabase Database	The system rejects unauthorized requests and prevents access to private workout records.	100% of requests using missing, invalid, or expired tokens are denied. Cross-account SELECT and INSERT attempts are also denied in the access-control tests.	QR-01
QAS-03	Maintainability	A developer submits a pull request containing changes to workout logging, authentication, or synchronization logic.	Flutter application codebase	Automated static analysis evaluates the complexity of modified methods and functions.	Cyclomatic complexity does not exceed 10 for any analyzed method or function before the pull request is merged.	QR-03

The architecture is shaped by three constraints. CON-01 requires compatibility with iOS and Android, supporting the selection of Flutter as a cross-platform development framework. CON-02 requires the protection of users' personal information and workout records, motivating authenticated access, account-specific SQLite storage, and Supabase row-level security policies. CON-03 prohibits misleading or unsupported health-related claims, so the application's progress graphs will present estimated exercise performance rather than medical or health assessments.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
A layered mobile architecture with local SQLite storage and a client-server connection to Supabase	This supports workout logging and retrieval of previously saved workout records (UC-01, UC-02), cross device access to workout history (UC-05), and separating responsibilities from maintenance (QR-03). The local SQLite storage allows users to access their saved workout records offline, and Supabase provides authentication and cloud synchronization.	It warrants additional development and maintenance of continuous cloud synchronization, resolving conflicts when records are changed on multiple devices, and ensuring separation between application layers.	Chosen
Cloud-Only Thin-Client Architecture	This architecture supports centralizing data management, user authentication, and cross-device access (UC-05). It also simplifies maintaining workout records consistently in a single cloud database.	The requirements for this architecture are internet connectivity for workout logging and retrieval. Additionally, network interruptions can prevent users from accessing their records, and the responsiveness of the application depends entirely on cloud service availability and latency of the network.	Rejected
Traditional three tier architecture with custom	It supports a clear separation between the user interface,	It requires the team to develop, deploy, secure, monitor, and maintain a custom	Rejected

backend / API server	application logic, and data management (QR-03), which improves maintainability. It also provides control over user authentication, workout data processing, and communication over system components.	backend server. This increases implementation effort, infrastructure costs, and operational responsibilities beyond the team's initial project scope.	

The team selected a layered mobile-client architecture with local-first SQLite storage and Supabase integration to balance performance, security, and maintainability. The local storage reduces reliance on network requests during application startup (QAS-01), whereas Supabase authentication and access controls support data security (QAS-02). This separation in application responsibilities preserves the maintainability of code complexity (QAS-03). Although this synchronization and conflict resolution introduce additional complexity, this approach best supports Rep Rank's requirements.

6.3 Component view

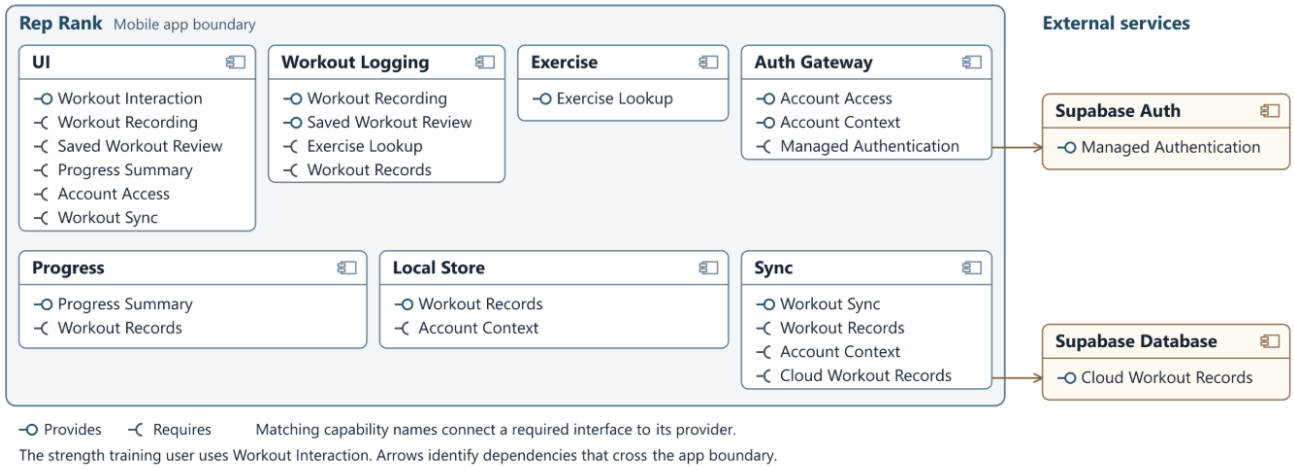


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
UI	Presents workout, progress and account flows and their results.	Workout Interaction	Workout Recording, Saved Workout Review, Progress Summary, Account Access, Workout Sync	UC-01, UC-02, UC-03, UC-04, UC-05, ADR-04
Workout Logging	Validates entries, records workouts and returns saved sessions and prior performance.	Workout Recording, Saved Workout Review	Exercise Lookup, Workout Records	UC-01, UC-02, UC-05, ADR-02
Exercise	Searches the bundled exercise records by name.	Exercise Lookup	None	UC-01
Progress	Calculates the highest estimated 1RM per exercise per date using FR-04.	Progress Summary	Workout Records	UC-03
Local Store	Saves workouts and pending sync state in account-scoped SQLite.	Workout Records	Account Context	UC-01, UC-02, UC-05, ADR-02
Sync	Exchanges account records with the cloud and reports sync status.	Workout Sync	Workout Records, Account Context, Cloud Workout Records	UC-05, ADR-02, ADR-03
Auth Gateway	Handles registration and sign-in and provides verified account context.	Account Access, Account Context	Managed Authentication	UC-04, UC-05, QAS-02, ADR-03
Supabase Auth (external)	Verifies credentials and issues sessions.	Managed Authentication	None	UC-04, UC-05, QAS-02, ADR-03
Supabase Database (external)	Stores workouts and enforces account isolation via row-level security.	Cloud Workout Records	None	UC-05, QAS-02, ADR-03

Workout Logging confirms a save after Local Store commits the entry and its pending sync state together. On a second device, the UI requests sync after sign-in, then displays the imported sessions through Saved Workout Review. If retrieval fails, the UI displays cached sessions with a warning that they may be out of date. When the device has no cached sessions, the UI reports that saved records could not be retrieved.

6.4 Deployment view

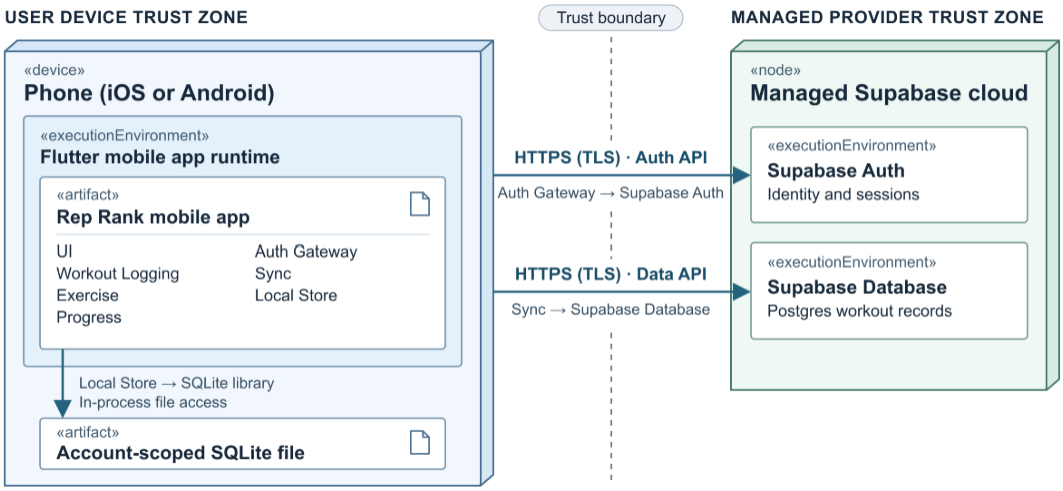


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
iOS or Android phone	Flutter runtime and Rep Rank app artifact containing all seven internal components. The app sandbox contains the account-scoped SQLite file.	Auth Gateway to Supabase Auth via HTTPS/TLS. Sync to the Supabase Data API via HTTPS/TLS. Local Store accesses the SQLite file through the in-process SQLite library.	User device and app sandbox
Managed Supabase project	Supabase Auth, the Data API and Postgres (Supabase Database).	Accepts the phone's two HTTPS/TLS connections. The Data API applies the account session to database requests.	Provider-managed cloud

Supabase operates the hosted services. The Rep Rank team configures and tests Postgres row-level security (RLS) for every exposed workout table, scopes local reads and writes to the verified account and protects session material on the phone. The app uses a publishable key with the user's session. Privileged keys belong in restricted server-side configuration. The team decides what it collects and retains, limiting cloud data to the account and workout fields required by the use cases.

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Exercise	Exercise	Represents an exercise that users can find and select through Exercise Lookup.
Exercise Entry	Workout Logging	Creates and validates entries. Progress derives estimated 1RM from their values.
Session / Workout Session	Workout Logging	Assembles the entries for a session. Local Store persists the session.
User	Auth Gateway	Owns the app's account identity. Supabase Auth handles password verification.
Account	Auth Gateway	Represents the signed-in User identity on the current device.
Workout Logging Controller	Workout Logging	Coordinates exercise lookup, reads prior performance through Workout Records and saves sessions.
Sign-In Controller	UI	Coordinates Account Access, Workout Sync and Saved Workout Review to display imported sessions in UC-05.
Authentication Service	Supabase Auth (external)	Implements credential verification through Managed Authentication.
Cloud Database	Supabase Database (external)	Supplies account-scoped records through Cloud Workout Records.

The Strength Training User is an external actor and uses the UI's Workout Interaction interface.

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Workout Logging confirms the local save while Sync handles later cloud transfer.	Workout Recording, Workout Sync, ADR-02
Information hiding	Auth Gateway encapsulates the provider's sign-in and token verification details.	Account Access, Account Context
Low coupling	The UI requests saved sessions through Saved Workout Review. Workout Logging reads through Local Store's Workout Records interface.	Saved Workout Review, Workout Records
High cohesion	Progress owns estimated 1RM calculation and selects the highest value per exercise per date.	Progress Summary, UC-03
Design for change	Sync handles changes to the cloud records API while Workout Logging continues to use Workout Records.	Cloud Workout Records, Workout Records, ADR-03
Keep it simple (KISS)	The app uses one local storage approach and managed services for authentication and cloud records.	Local Store, Supabase Database, ADR-01

7 Design Rationale

7.1 Architecture decision records

ADR-01: Selection Of Overall Architectural Style

Status	Accepted
Date and owner	November 7 , 2026 – Shreya Singaravel Sabapathi
Context (drivers)	Rep Rank requires reliable workout logging (UC-01), retrieval of previously recorded workouts (UC-02), and access to workout records across devices (UC-05). The architecture also must support maintainability (QR-03), cross platform compatibility (CON-01), and protection of user information (CON-02).
Options considered	Option 1: Layered mobile client architecture with local SQLite storage and Supabase cloud services. Option 2: Cloud only thin-client architecture with centralized storage Option 3: Traditional three tier architecture with a custom backend/API server
Decision	The final decision is to adopt a layered mobile-client architecture using SQLite for local workout storage and Supabase for authentication and cloud synchronization. The mobile application will separate user interface functionality, application logic, and data management responsibilities into distinct layers.
Consequences	This architecture supports local access to workout records, cross device synchronization, and maintainability through separation of responsibilities. However, it introduces additional complexity in maintaining local and cloud data consistency, resolving synchronization conflicts, and coordinating communication between application layers.
Evidence	The P1 Flutter and SQLite prototype in dev/spike/flutter_sqlite_demo provides preliminary implementation evidence for the proposed mobile and local storage approach.
Related	ADR-02, ADR-03, ADR-04, UC-01, UC-02, UC-05, QR-03, CON-01, CON-02

ADR-02: Local-First Storage vs. Cloud-Only Storage

Status	Accepted
Date and owner	November 8, 2026 – Shreya Singaravel Sabapathi
Context (drivers)	Rep Rank requires users to record workout information (UC-01) , retrieve previously saved workout records (UC-02), and access workout history across multiple devices (UC-05). The application must support reliable workout tracking even when network connectivity isn't available. The storage architecture must also protect confidential personal workout records (CON-02).
Options considered	Option 1: Local-first architecture using SQLite for on-device storage with synchronization to Supabase. Option 2: Cloud-only architecture that stores and retrieves workout records directly from Supabase without a persistent local workout database.
Decision	The decision was to adopt a local-first storage approach using SQLite to store workout records directly on the user's device. When the network connection is

	present, workout information will be synchronized automatically with Supabase to support backup and cross-device access. Each record carries a last-modified time. When the same record has been changed on two devices, the later change is kept and the other is discarded. Records added on different devices are all kept. This rule is deliberately simple and will be tested with two devices in P3.
Consequences	This approach helps improve offline availability by allowing users to record and retrieve locally stored workout information without continuous internet connectivity. However, it also introduces extra development complexity including synching failures, conflicts in updates across multiple devices, and maintaining consistency between local and cloud databases. Last-write-wins can silently discard an earlier edit.
Evidence	The P1 Flutter and SQLite prototype in dev/spike/flutter_sqlite_demo provides preliminary implementation evidence for local storage approach. Spike S-01 (section 8) showed bidirectional sync of simple records between account-specific SQLite and Supabase, and recovery of records after a failed upload. It did not test conflicting edits from two devices or the relational workout schema; both are P3 work (Q-02).
Related	ADR-01, ADR-03, UC-01, UC-02, UC-05, FR-01, FR-02, FR-09, CON-02, R6.

ADR-03: Selection Of Backend Service Provider

Status	Accepted
Date and owner	November 8, 2026 – Shreya Singaravel Sabapathi
Context (drivers)	Rep Rank requires secure user authentication (FR-07, FR-08), storage of account specific workout records, and synchronizing workout history across devices (FR-09). The team has limited development resources and does not intend to build and maintain its own authentication and cloud storage infrastructure, as established in Section 1.2. The selected service must also support the application's security and privacy requirements (QR-01, CON-02).
Options considered	Option 1 - Supabase : Provides managed PostgreSQL database services, authentication, and APIs for application integration. Option 2 -Firebase: Provides managed authentication and cloud database services through Google's backend platform, with a different data modeling approach. Option 3 – Custom Backend Server: Develop and maintain separate backend API, database infrastructure, and authentication functionality.
Decision	The decision was to select Supabase as the backend service provider for user authentication, cloud database storage, and synchronization of workout records. The PostgreSQL database supports the application's purpose of structuring workout records, and its authentication services reduce the need to develop custom account management infrastructure. Firebase was rejected because its document model fits the relational workout schema less well; a custom backend was rejected because the team cannot build and secure one within the course.
Consequences	Supabase reduces the cost of backend development and infrastructure maintenance responsibilities. It also supports authentication and centralized data storage. However, the application also becomes dependent on Supabase's service availability, pricing, supported features, and security configuration. The team will remain responsible for configuring access controls and integrating a secure service with the mobile application.

Evidence	The project's scope in Section 1.2 establishes the use of third-party authentication and cloud storage. Supabase's service and pricing documentation [R10] supports the provider evaluation. The team's P2 architecture spike (S-01) provided preliminary evidence of cloud synchronization and account-specific access controls.
Related	ADR-01, ADR-02, UC-05, FR-07, FR-08, FR-09, QR-01, CON-02, R2, R7, R8.

ADR-04: Selection of Cross-Platform Development Framework

Status	Accepted
Date and owner	November 8, 2026 – Shreya Singaravel Sabapathi
Context (drivers)	Rep Rank must support both iOS and Android operating systems (CON-01). The development team requires support in consistent application functionality, manageable maintenance, and efficiently implementing across both platforms. The selected framework must also support integration with local SQLite storage and third-party authentication and database services.
Options considered	Option 1 - Flutter: A cross-platform development framework using Dart that supports building iOS and Android applications from a shared codebase. Option 2 - React Native: A cross-platform framework using JavaScript or TypeScript that also supports shared mobile development. Option 3 - Native Development: Developing separate iOS and Android applications using platform specific technologies such as Swift and Kotlin.
Decision	The decision was to select Flutter as the mobile development framework for Rep Rank. A shared Flutter codebase will be used to develop the iOS and Android applications as it supports consistent functionality and reduces the need for duplicating development efforts. React Native was rejected because the team's P1 prototype and spike were already built in Flutter.
Consequences	This approach supports cross-platform compatibility and maintainability as it allows the development team to implement most application functionality within a single codebase. However, platform specific adjustments, additional testing, and native integrations may still be required. The team must also maintain compatibility of the app with the selected SQLite and Supabase integration packages.
Evidence	The P1 Flutter and SQLite prototype provides evidence for integrating Flutter with local workout storage. The decision also supports the cross platform requirement defined in CON-01. Spike S-01 (section 8) confirmed that Supabase Auth and RLS SELECT and INSERT policies separated two test accounts. Access control on the full workout schema is still to be verified in P3 (Q-03)
Related	ADR-01, ADR-02, ADR-03, CON-01, QR-02, QR-03, R4.

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability and Modularity	Adopt a layered mobile client architecture that separates the user interface, application logic, and data	Required additional coordination between application layers and clearly defined interfaces, increasing	ADR- 01

	management responsibilities.	initial design and integration effort.	
Offline availability and reliability	Implement local first SQLite storage for workout records, with synchronization to Supabase when network connectivity is available.	Introduces additional complexity in synchronizing local and cloud records, handling synchronization failures, and resolving conflicting updates across devices.	ADR-02
Security and development efficiency	Use Supabase for third party authentication and cloud database management instead of developing and maintaining a custom backend server.	Creates a dependency on Supabase's availability, service limitations, pricing , and security configuration, reducing direct control over backend infrastructure.	ADR-03
Cross platform compatibility and maintenance	Use Flutter to develop a shared mobile application codebase for iOS and Android.	May require platform specific adjustments , additional testing , and integration work to ensure consistent functionality across both operating systems.	ADR-04

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 — Flutter SQLite and Supabase Authentication, Synchronization, and Access Control
Risk or decision it addresses	R2, R3 / ADR1, ADR2, ADR3 / QAS-2
Question (answerable yes or no)	Can Flutter support multiple authenticated users with separate local SQLite databases, synchronize records bidirectionally with Supabase, and enforce user-specific access controls through Row Level Security (RLS)?
Set-up	Developed a Flutter application using SQLite for local storage and Supabase for authentication and cloud storage. Each authenticated user has a separate local SQLite database. Notes are stored locally and uploaded to Supabase using a manual synchronization button. Cloud records are downloaded to SQLite upon login. The application was tested on Windows and an Android emulator using two separate user accounts.
How it was measured	• Tested account creation, login, logout, and separation of local data between two accounts. • Tested uploading and downloading records between SQLite and Supabase. • Simulated a failed upload by disconnecting from the internet and verified that unsynchronized records were preserved and could be uploaded afterward. • Tested synchronization behavior to prevent duplicate records. • Tested RLS SELECT restrictions by attempting to retrieve another user's records. • Tested RLS INSERT restrictions by attempting to create records belonging to another user.
Result (numbers)	• 2 user accounts created and tested. • 2 execution environments: Windows and Android emulator. • 2 synchronization directions implemented: SQLite → Supabase and Supabase → SQLite. • 2 RLS policies tested: SELECT and INSERT. • SELECT access-control scenario passed. • INSERT access-control scenario passed. • Failed upload recovery test passed. Answer to the question is yes Flutter can support multiple authenticated users with separate local SQLite databases, synchronize records bidirectionally with Supabase, and enforce user-specific access controls through Row Level Security (RLS).
What it changed or confirmed	Confirmed that Flutter can support account-specific SQLite storage and bidirectional synchronization with Supabase for simple string records. Testing also demonstrated that Supabase RLS policies successfully restricted unauthorized SELECT and INSERT operations. These results support ADR1 and ADR2 and reduce the identified authentication, synchronization, and access-control risks. The spike does not establish the reliability of synchronization for the more complex relational workout database.

Where to see it	RLS Insert and Select testing:https://github.com/AkshayaKomaragiri/software-engineering-project/issues/15#issue-6268917437, Branch : https://github.com/AkshayaKomaragiri/software-engineering-project/tree/dev/spikes Application used for account authentication and SQlite to Supabase to SQlite synchronization: dev/spike/flutter_supabase_sync

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-01	Session, Exercise Entry, Exercise	Workout Logging, Local Store	ADR-01, ADR-02	UC-01 analysis (4.2); S-01 demonstrates local storage with simple records; workout logging pending P3.
FR-02	UC-01, UC-02	User, Session, Exercise Entry	Workout Logging, Local Store	ADR-02	UC-01 and UC-02 analysis (4.2); S-01 demonstrates local persistence of simple records.
FR-03	UC-01	Exercise, Exercise Entry	Workout Logging, Local Store	ADR-02	UC-01 step (6); previous performance retrieval pending implementation.
FR-04	UC-03	Exercise, Exercise Entry, Session	Progress, Local Store, UI	ADR-01	FR-04 defines the estimated 1RM formula; graph implementation pending P3.
FR-05	UC-06	User	UI	ADR-01	Optional weekly workout goal; implementation pending P3.
FR-06	None (WON'T)	None	None	None	Competitive leaderboard excluded from initial scope (1.2).
FR-07	UC-04	User	Auth Gateway, Supabase Auth	QAS-02, ADR-03	S-01 reports account creation with two test accounts.
FR-08	UC-05	User	Auth Gateway, Supabase Auth	QAS-02, ADR-03	UC-05 analysis; S-01 reports authentication and RLS tests.
FR-09	UC-05	User, Session, Exercise Entry	Sync, Local Store, Supabase Database	ADR-02, ADR-03	S-01 reports bidirectional synchronization of simple records.
FR-10	UC-01	Exercise	Exercise, Workout Logging	ADR-01	UC-01 steps (3)-(5); exercise search implementation pending P3.
QR-01	UC-01, UC-02, UC-05	Not applicable	Auth Gateway, Supabase Auth	QAS-02, ADR-03	S-01 reports RLS tests; complete invalid/expired token verification pending.
QR-02	UC-01, UC-02, UC-05	Not applicable	UI	QAS-01, ADR-04	Cold-start performance testing pending P3.
QR-03	UC-01, UC-02, UC-05	Not applicable	All internal components	QAS-03, ADR-01	Automated complexity analysis pending P3.
QR-04	UC-01, UC-02	Not applicable	UI	ADR-04	Optional orientation testing pending P3.
QR-05	UC-01, UC-02	Not applicable	UI	ADR-04	Optional touch-target testing pending P3.

CON-01	All applicable use cases	Not applicable	UI, mobile components	ADR-04	S-01 tested Windows and Android emulator; iOS testing pending.
CON-02	UC-04, UC-05	User	Auth Gateway, Local Store, Supabase Database	QAS-02, ADR-03	S-01 reports account isolation and RLS SELECT/INSERT tests.
CON-03	UC-03	Exercise Entry	Progress, UI	ADR-01	Estimated 1RM is presented as a progress indicator; claims review pending P3.

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	FR-06 (WON'T) has no assigned component because the competitive leaderboard is excluded from the initial release. All other functional requirements are mapped to architectural components.	Retained FR-06 in the traceability matrix to preserve scope decisions. Cross-checked the remaining requirements against the component responsibilities in Section 6.3.
Components with no requirement, QAS or ADR	No unmapped components were identified. The UI, Workout Logging, Exercise, Progress, Local Store, Sync, Auth Gateway, Supabase Auth, and Supabase Database components support the requirements, use cases, quality-attribute scenarios, or architectural decisions.	Cross-checked every component in Section 6.3 against the functional requirements, quality-attribute scenarios, and architectural decisions. All components support at least one documented requirement, use case, QAS, or ADR, so no additional component mappings are needed.
Use cases not analyzed (and why that is acceptable)	UC-03 (View Exercise Progress), UC-04 (Create Account), and UC-06 (Set Weekly Workout Goal) were not selected for detailed analysis.	Detailed analysis focuses on UC-01, UC-02, and UC-05 because they represent the core logging, history retrieval, and cross-device synchronization workflows. The remaining use cases are retained in the requirements and use-case model for future implementation.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Anshika Anshika	Backend Development and Cloud Integration Lead: Responsible for cloud database development, backend integration, and synchronization between local SQLite storage and Supabase.	Team discussion, followed by majority vote, if consensus cannot be reached.
David Wittler	System Analyst: Work with developers to ensure the UI, database, authentication, and synchronization components interact correctly. Define how user actions, such as recording or editing a workout, translate into database operations. Finalize the privacy policy based on actual data-handling practices, validate user access controls and account deletion processes, and verify that implemented functionality satisfies system requirements.	Team discussion, followed by majority vote, if consensus cannot be reached.
Laxmi Akshaya Komaragiri	Database Architecture & Security: Responsible for designing the Supabase cloud schema, setting up relational constraints, and configuring row-level security policies to ensure data isolation.	Team discussion, followed by majority vote, if consensus cannot be reached.
Shreya Singaravel Sabapathi	Local Database and Analytics Lead: Responsible for implementing account - specific SQLite storage, ensuring secure storage and retrieval of workout records, and developing workout history and progress visualization graphs to track users' exercise performance over time.	Team discussion, followed by majority vote, if consensus cannot be reached.
Tanishq Seetala	Frontend and UI/UX Lead: Responsible for designing and implementing the mobile application user interface which includes workout logging screens, exercise selection, progress graphs, dashboard and navigation. Ensure the interface is intuitive, responsive, and compatible with both iOS and Android platforms.	Team discussion, followed by majority vote, if consensus cannot be reached.
Veda Manchana	Authentication and cross-platform testing lead. Responsible for implementing user registration, login, logout, and password recovery, as well as testing core application features on iOS and Android.	Team discussion, followed by majority vote, if consensus cannot be reached.

Team Communication and Meetings: The team communicates through Discord and meets on Sunday at 6pm CST to discuss project progress, resolve blockers, and coordinate upcoming tasks. Decisions are made through team discussion, followed by a majority vote if consensus cannot be reached.

10.2 Plan to P3

Milestone	Date	Owner	Done when

1.) SP-01 completed and tested. This will set the base for the SQLite and supabase synchronization, including support for unique users.	11/8/2026	David Wittler	SP-01 is a runnable application that can be tested across multiple devices. The details for this spike are in section 8.
2.) Develop Core Workout Logging UI Develop the workout session interface, including adding exercises, recording sets, and navigating between current and previous sessions.	11/11/2026	Tanishq Seetala	It is established that adding a workout session opens up a new screen, where one can then add exercises, where they can add sets, then add reps and weight. Where progress graphs will go is clearly defined and were setting up a weekly goal will go is clearly defined. The UI is laid out with where these features will be implemented in mind
3.) Independently build a predefined exercise catalog and a flutter search interface for this catalog.	11/23/2026	Tanishq Seetala	A search bar application is completed. One can run this application, type in the search bar, and see exercises that most likely match the search they entered. In the background, an exercise table should be built associating exercise IDs to each exercise in the predefined catalog.
4.) Replace the simple string-entry table in Sp-01with the actual workout schema: workout sessions, exercise logs, sets, and exercise references. Build the SQLite CRUD operations around that schema. Update the Supabase schema and synchronization logic accordingly. Implement UI developed in 2.)	11/23/2026	Shreya Singaravel Sabapathi , Anshika Anshika and Laxmi Akshaya Komaragiri	A separate application built off S-01 is completed. Instead of the user entering strings, they can enter workout sessions. Logs will show previously entered workout sessions instead of strings. This application will be the first prototype of Rep Rank; further development will work directly on this.
5.) Test sync and access controls across the cloud and test that one account can't read records from another account on the new working Rep Rank prototype (not S-01).	11/25/2026	Veda Manchana	The working prototype for Rep Rank avoids risks R2 and R3 as defined in section 10.3.
6.) Integrate the independently developed exercise catalog (4.) and search interface (3.) into the Rep Rank prototype.	11/27/2026	Tanishq Seetala and Anshika Anshika	Somebody using Rep Rank, can now enter an exercise name, and in doing so search for the exercise, when creating exercise entries in a workout session. Before this, workouts will be picked from a list of workout IDs or names.
7.) implement progress graphs. Display the users maximum estimated 1RM for each date that their 1RM on the exercise is updated (each date they do that exercise).	12/01/2026	Shreya Singaravel Sabapathi	A separate screen will be dedicated to the progress graphs and a user can understand their progress over time when looking at these graphs.
8.) Implement Weekly Workout Goals Develop functionality allowing users to set and modify weekly workout goals.	12/02/2026	David Wittler	Users can establish a target number of workout sessions per week, modify their target, and view their progress toward completing it. The application automatically updates progress as workout sessions are completed and resets weekly progress at the beginning of each new week while preserving the user's goal.
9.) Final UI Refinement and Integration Testing Finalize navigation, ensure consistent layouts, and verify that workout logging, progress graphs, and weekly goals function correctly together.	12/07/2026	Tanishq Seetala	QR04 and QR05 will be satisfied. The placements of all widgets, and where screen changes will occur will be finalized.

10.) Security, Privacy, and release preparation: Finalize privacy policy based on actual data handling. • Supabase access controls prevent unauthorized access to another user's records. (checked in SP-01 and 4.)) • Logging out prevents the next user from viewing the previous user's local records through the application. (checked in SP-01 and 4.)) • Account and data deletion requirements are supported. • The privacy policy accurately reflects the application's actual data handling.	12/9/2026	Veda Manchana	Users must acknowledge and accept the privacy policy before using the application. The privacy policy accurately reflect the application's data collection, storage, usage, and sharing practices and comply with applicable privacy laws and regulations (CON - 02)

Note: 3.) and 4.) can be worked on concurrently by different team members. The biggest concern in development is verifying the synchronization of the local database and cloud database and vice versa, and also user unique user support. This pathway of milestones prioritizes that first.

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1	Open. P2 has several hand-offs between sections	Moderate/High	David Wittler: section due dates and review pairs set in the P2 team plan; overdue sections reassigned at the Saturday check-in.
R2	Open, likelihood lowered to Moderate. Spike S-01 (section 8) demonstrated sign-in, two-account separation and bidirectional sync for simple records. It has not been tried on the full workout schema.	Moderate	Veda Manchana: spike S-01 tests sign-in, sync and access control against the cloud service with measured results (section 8).
R3	Open, likelihood lowered. Spike S-01 passed the RLS SELECT and INSERT tests with two accounts (section 8). The tests must be repeated on the full workout schema in P3.	Moderate	Veda Manchana: spike S-01 tested this with two accounts; add the result as a number, for example 0 of N cross-account reads returned. Re-test on the workout schema in P3 (10.2, milestone 5)
R4	Open; moved to P3, since no app features exist yet to test.	Moderate/Moderate	Veda Manchana: test each P3 feature on both platforms and record issues in GitHub. One shared codebase (ADR-04) reduces the difference.
R5	Open, likelihood lowered. Change control is in use: every change since P1 is recorded and approved in 0.2.	Moderate (was High) / High	Shreya Singaravel Sabapathi: every new requirement needs a 0.2 change row and team approval before work starts.
R6 (new)	New, created by the local-first design	Moderate / High	Shreya Singaravel Sabapathi: ADR-02 records the conflict rule. Anshika Anshika: test two-device edits in P3.

R7	New	Moderate / High	Veda Manchana: spike S-01 and QAS-02. Laxmi Akshaya Komaragiri: review the access rules before each P3 release.
R8	New	Moderate / Moderate	Laxmi Akshaya Komaragiri: record the limits in ADR-03 and monitor usage. Local-first storage keeps logging working while the cloud is unavailable.
R9	New	Moderate / Moderate	David Wittler: PR reviews check new code against the 6.3 component view. Tanishq Seetala: update 6.3 when a design change is approved.

10.4 Repository

Repository URL: https://github.com/AkshayaKomaragiri/software-engineering-project. The repository is public.

Folder	Contents
/docs/p2	This P2 document as a PDF, on the commit tagged p2
/docs/p2/diagrams	Source files and exported images for every P2 diagram (use cases, domain model, sequence, component, deployment)
/spikes	P2 architecture spike S-01: code, README and measured results
/docs	Each member's contribution records (p1-…, p2-…-contributions.md), the P1 use-case diagram, and screenshots in /docs/images
/dev/spike/flutter_sqlite_demo	P1 Flutter and SQLite spike, cited as evidence in ADR-02
Repository root	README and the P1 PDF

Branch and pull-request rule (carried from P1 4.4, with P2 additions)

- Each member works on their own branch, named `dev/<feature name>`, `update/<feature name>/<update type>` or `bugfix/<fix name>`. P2 documentation branches are named `dev/p2-documentation-<member>`.
- Every commit message starts with `feat:`, `fix:`, `docs:`, `refactor:` or `test:`.
- Nothing is pushed directly to main. Every change is merged through a pull request reviewed by a member other than its author.
- The P1 documentation branches were merged into main in response to the P1 feedback.
- Passwords, API keys and other credentials are never committed. Local configuration (`.env`) is excluded by `.gitignore`.
- Tasks are tracked on the GitHub project board: backlog, ready, in progress, in review, done.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
David Wittler	Chat GPT	10.2 Milestone	Checked what the weekly workout goal setup should look like.
Tanishq Seetala	Codex	Formatting 6.3 -6.6	Checked if every detail still remained
Shreya Singaravel Sabapathi	Chat GPT	Used to refine in Sections 6.2, 7.1, and 7.2.	Reviewed the output against the P2 template, project requirements, and team architecture.
Veda Manchana	ChatGPT	Drafting and refining Sections 6.1, 9.1, 9.2, 11	Verified against requirements and architecture.
Laxmi Komaragiri	Claude	sections 4.1 and 4.2	Used to refine text to improve conciseness and clarity
Anshika Anshika	Claude	Reviewing the draft (0.2, 4.1(revised), 4.2(revised), 4.3, 10.3, 10.4) against the rubric and P1 feedback	Checked each change against the repository and rubric; kept only what is true

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q-01	Can Flutter support local SQLite storage and synchronization with Supabase across devices?	Development team	David Wittler - Nov. 23, 2026	Partially resolved. Spike S-01 demonstrated bidirectional synchronization of simple records on Windows and an Android emulator. Synchronization of the complete workout schema remains unverified.	The same approach can be extended to workout sessions and exercise entries during P3.
Q-02	How will synchronization conflicts be handled when workout records are changed on multiple devices?	Development team	Anshika and Shreya - Nov. 25, 2026	Partially resolved. ADR-02 sets a last-write-wins rule. Two-device behavior is not yet tested (R6).	The rule in ADR-02 applies until P3 testing shows a problem.
Q-03	Can Supabase authentication and row-level security prevent users from accessing workout records belonging to another account?	Development team	Veda and Laxmi - Nov. 25, 2026	Partially resolved. Spike S-01 successfully tested RLS SELECT and INSERT restrictions using two accounts. Access controls must be verified again using the complete workout schema.	Account-specific local storage and Supabase RLS policies will continue to enforce user separation, subject to further testing.
Q-04	Will Rep Rank achieve its cold-start performance target on supported iOS and Android devices?	Development team	Tanishq – Dec. 7, 2026	Open. QR-02 defines a cold-start target below 2.0 seconds, but spike S-01 does not report cold-start measurements.	The target remains a performance requirement to be evaluated during P3 testing.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Anshika Anshika	AA	11/8/26
Tanishq Seetala	TS	11/7/26
David Wittler	DW	11/7/26
Veda Manchana	VM	11/8/26
Shreya Singaravel Sabapathi	SSS	11/8/26
Laxmi Akshaya Komaragiri	LAK	11/8/26